



Estruturas LIFO e FIFO

Fundamentos e Implementação
de Pilhas e Filas

Disciplina: Estrutura de Dados 2

Docente: Professora Kadidja Valéria

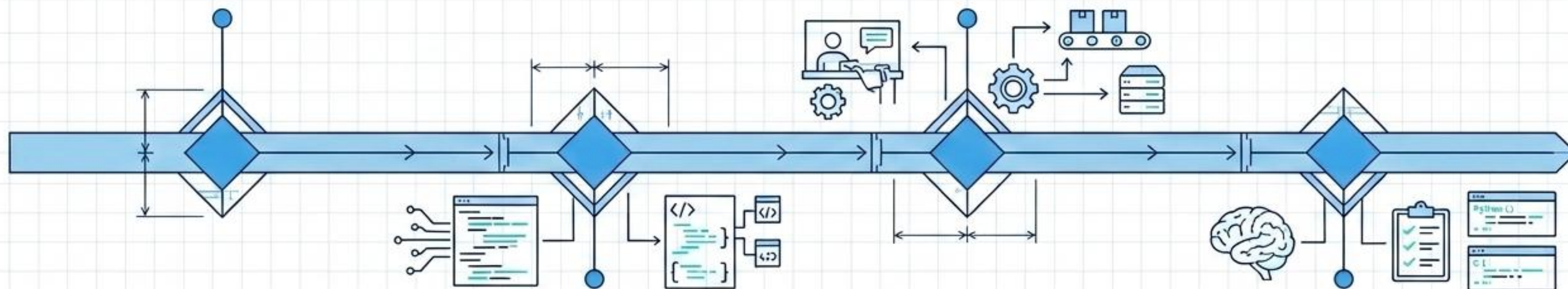
Roteiro da Aula

01 | Teoria e Metáforas (15 min)

LIFO vs FIFO.

03 | Laboratório em Grupo (40 min)

Resolução de problemas do mundo real.



02 | Implementação em Python (30 min)

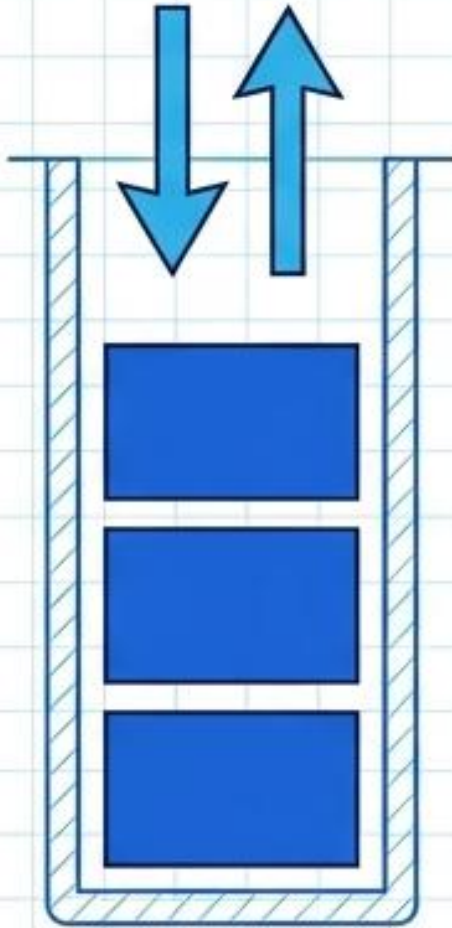
Mapeamento de código e estruturas nativas.

04 | Fixação e Reflexão (10 min)

Exercícios focados em Python e C.

Competências Desenvolvidas: Desenvolvimento de algoritmos, resolução de problemas, abstração estrutural e implementação orientada a aplicações.

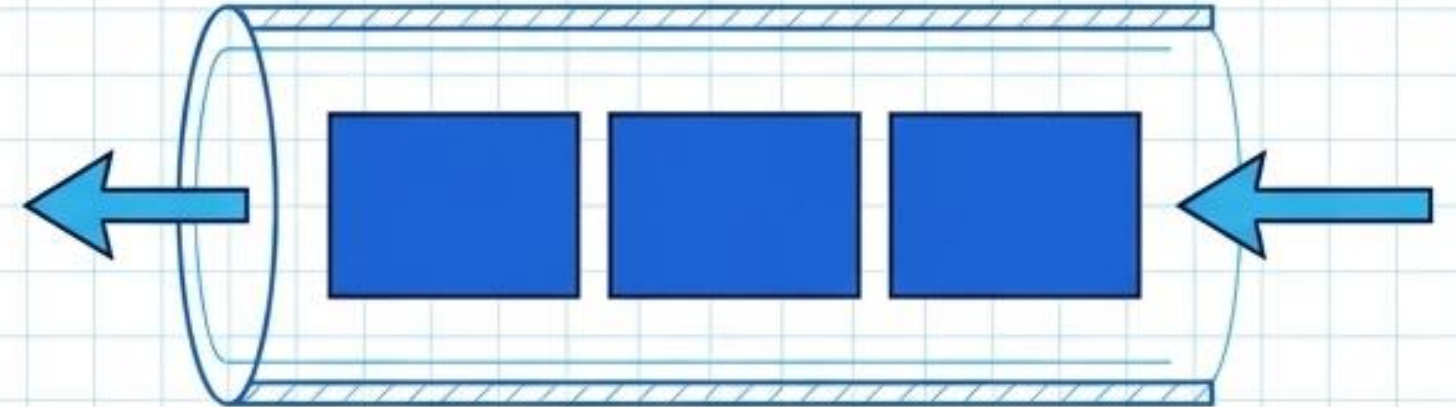
PILHA (LIFO)



Last In, First Out

O último elemento inserido é o primeiro a ser removido.

FILA (FIFO)



First In, First Out

O primeiro elemento inserido é o primeiro a ser removido.

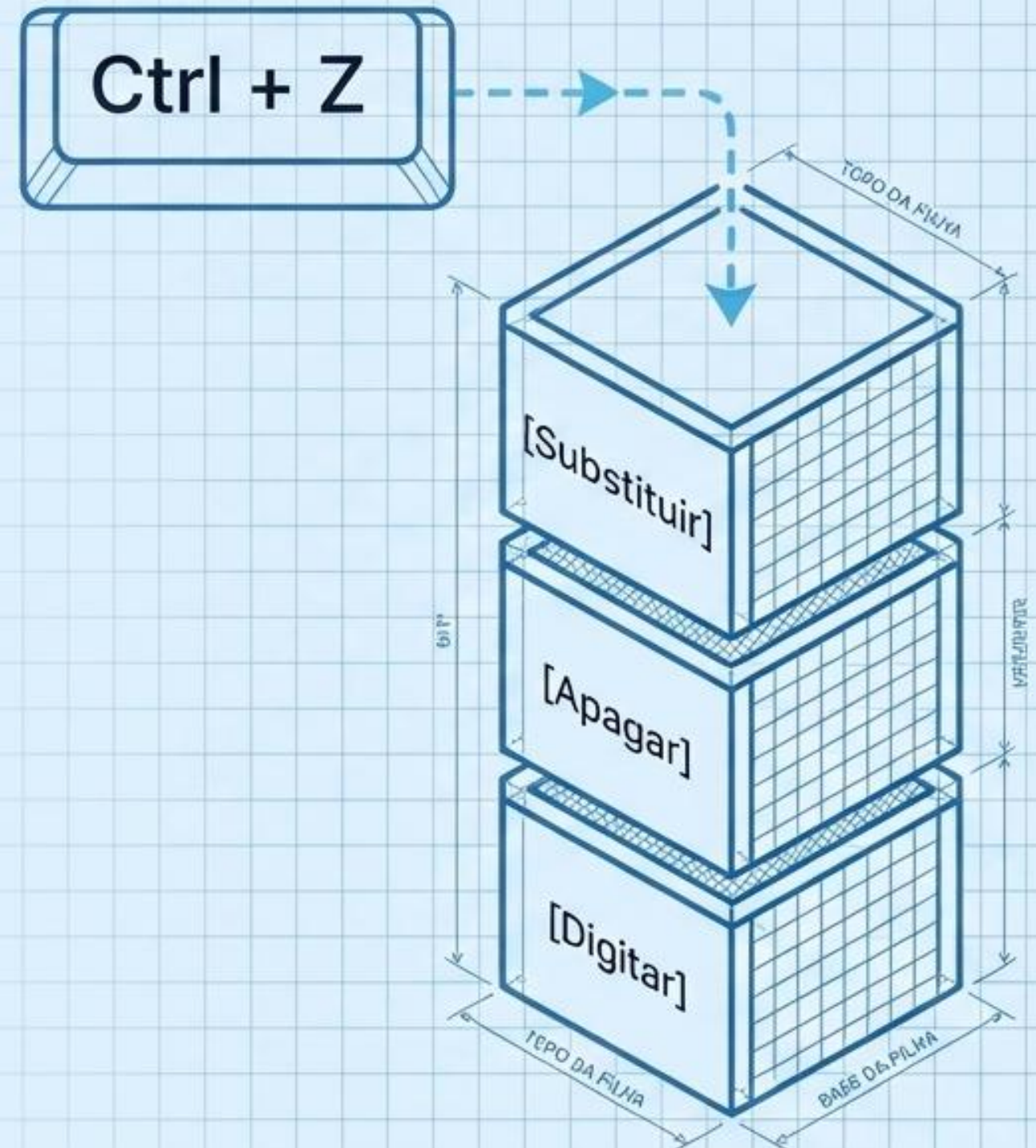
Pilha (LIFO)

O que é?

Uma estrutura de dados restrita onde a inserção e a remoção de elementos ocorrem estritamente por uma única extremidade, chamada de 'topo'.

Aplicações Reais no Software:

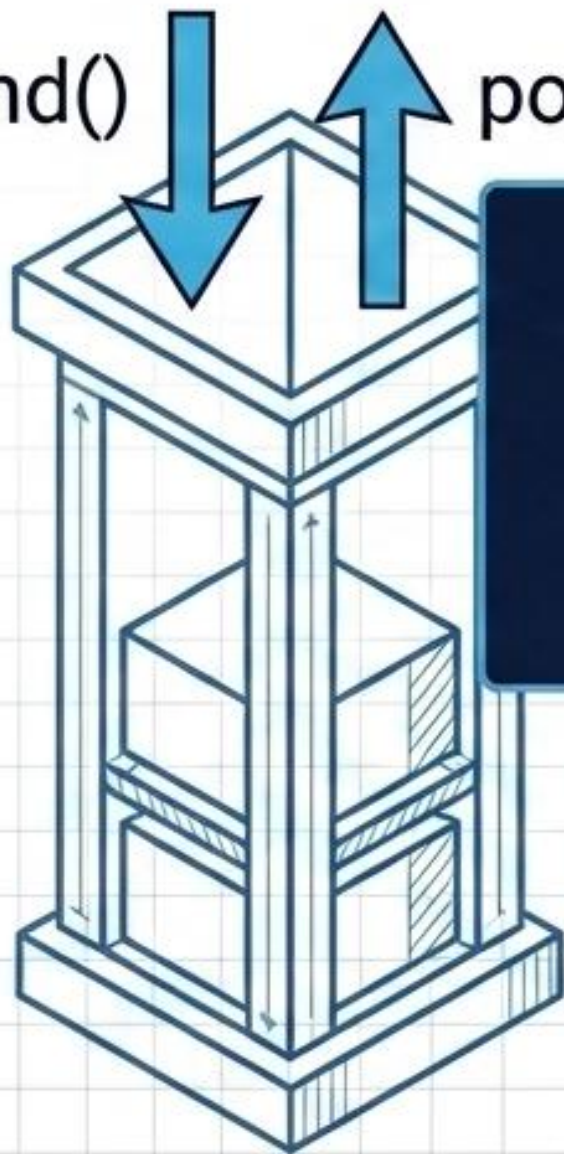
- Sistema de 'Desfazer' (Undo) em editores de texto.
- Navegação de histórico em navegadores web (botão 'Voltar').
- Gerenciamento de chamadas de função na memória (Call Stack).



Estrutura em Python: A Pilha

Ferramenta Nativa: Em Python, utilizamos a estrutura clássica de listas para simular pilhas com alta eficiência de ponta.

append() ↑ pop()



```
pilha = []  
pilha.append('A') # Insere no topo  
item = pilha.pop() # Remove do topo
```

Regra: O método `append()` adiciona ao final (que atua como o topo da pilha), e `pop()` sem argumentos remove e retorna esse exato último elemento.

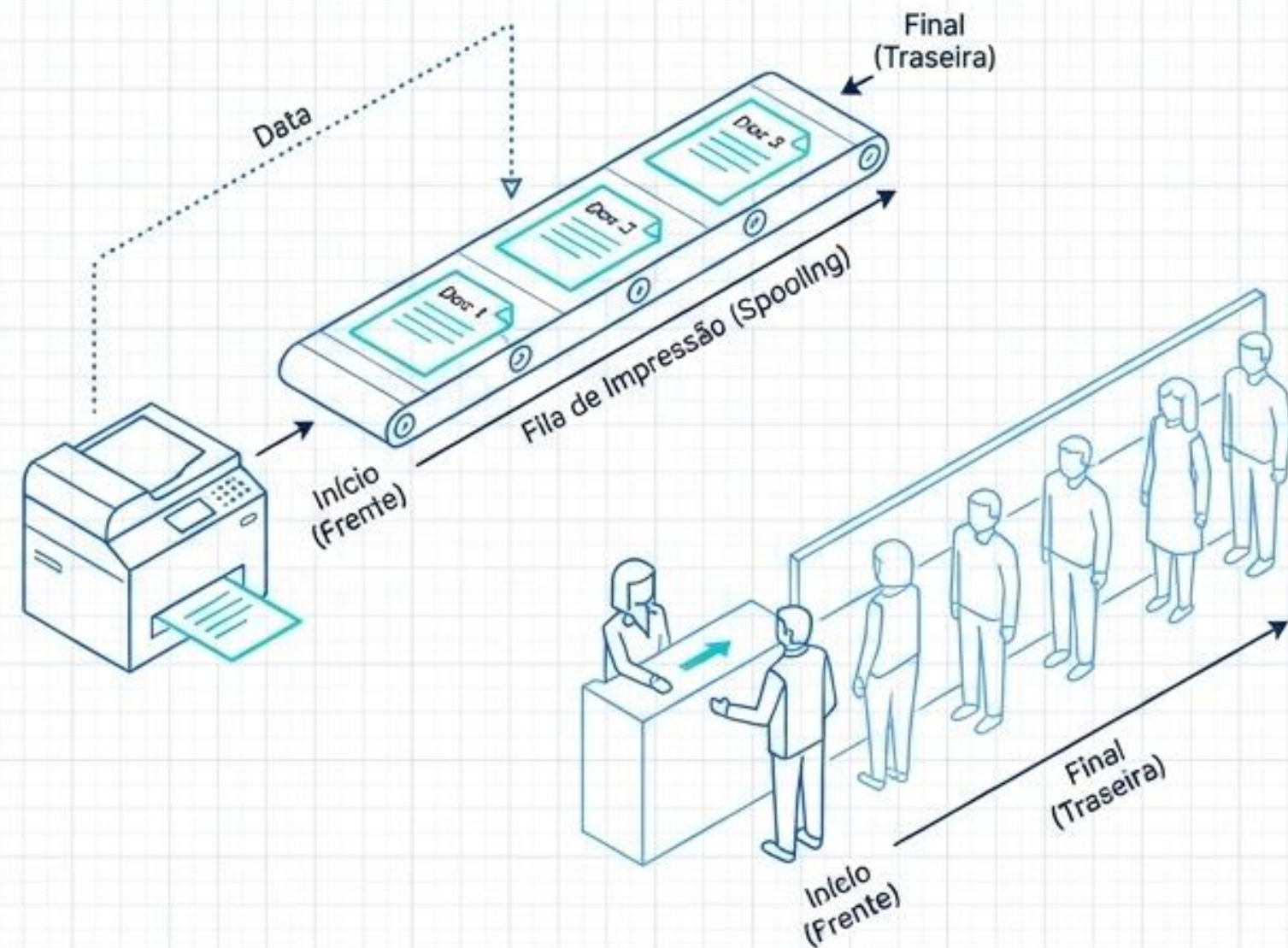
Anatomia de uma Fila (FIFO)

O que é?

Uma estrutura sequencial onde as inserções ocorrem exclusivamente no final (traseira) e as remoções ocorrem exclusivamente no início (frente).

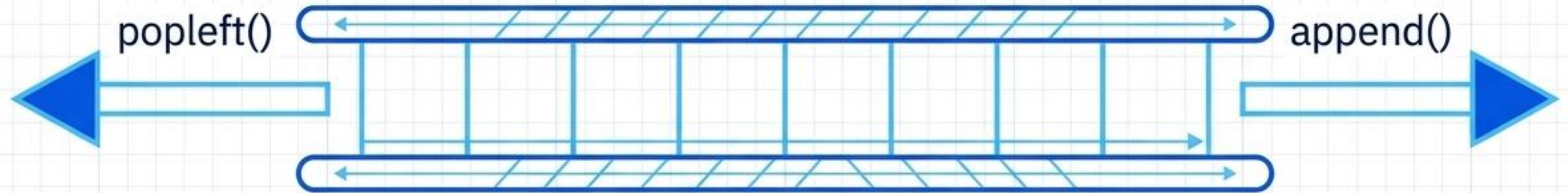
Aplicações Reais no Software:

- Fila de impressão de documentos em rede (Spooling).
- Sistemas de senhas para atendimento bancário.
- Buffer de dados em transmissão de áudio/vídeo (Streaming).



Estrutura em Python: A Fila

Ferramenta Otimizada: Utilizamos `collections.deque` (Double-Ended Queue).



```
from collections import deque
fila = deque()
fila.append('Doc_1') # Insere no final
item = fila.popleft() # Remove do início
```



Aviso de Performance: Listas comuns em Python são ineficientes para filas, pois remover do início (`pop(0)`) exige o deslocamento de todos os outros elementos na memória!

LIFO vs FIFO

Critério	Pilha (LIFO)	Fila (FIFO)
Comportamento	Último a entrar, Primeiro a sair	Primeiro a entrar, Primeiro a sair
Fluxo de Dados	Reversão / Retrocesso	Sequencial / Ordem de chegada
Método de Inserção	<code>append()</code> (no topo)	<code>append()</code> (no final)
Método de Remoção	<code>pop()</code> (do topo)	<code>popleft()</code> (do início)
Estrutura Ideal (Python)	<code>list</code> nativa	<code>collections.deque</code>

LABORATÓRIO PRÁTICO EM GRUPO (40 MIN)

Transformando abstração estrutural em algoritmos aplicados.



Regra: Para os próximos desafios, analise o problema, identifique o fluxo de dados exigido e escolha a arquitetura correta (LIFO ou FIFO) antes de escrever a primeira linha de código.

Desafios de Fundamentos

Exercício 1 | O Simulador "Desfazer"

Objetivo: Implemente uma Pilha para gerenciar ações em um editor de texto.

Requisitos: Armazene cada ação (digitar, apagar, substituir). Ao acionar a função 'desfazer', o programa deve remover e reverter estritamente o último comando inserido.

Exercício 2 | O Sistema de Impressão

Objetivo: Implemente uma Fila para gerenciar um spooler de impressora.

Requisitos: Cada novo documento é enfileirado. O sistema deve garantir que o documento impresso seja sempre o mais antigo aguardando na fila.

Desafio Master 🚀 | Triagem Hospitalar



Cenário: Criar um programa de gerenciamento para uma fila de atendimento médico.

Regras de Negócio:

- **Fluxo Padrão:** Pacientes normais entram no final da fila.
- **Fluxo de Exceção:** Pacientes prioritários (idosos, emergências) devem furar a fila e entrar diretamente no início do atendimento.



Dica de Arquitetura: A estrutura `collections.deque` é uma fila de duas pontas. Explore os métodos de `inserção à esquerda` para resolver a prioridade sem reescrever a fila inteira!

Exercícios de Fixação | Trilha Python



01 | A Pilha Matemática

Missão: Crie uma pilha (LIFO) que armazene números inteiros. Escreva um algoritmo que esvazie a pilha iterativamente, calculando e retornando a soma total dos elementos removidos.

02 | O Call Center

Missão: Crie uma fila (FIFO) que armazene nomes de clientes. Simule um ciclo contínuo de atendimento: adicionar novos clientes à espera e "chamar" o próximo cliente da fila para atendimento.

Exercícios de Fixação | Trilha C (Baixo Nível)



01 | Pilha via Array Estático

Missão: Implemente uma pilha em C alocando um array de tamanho fixo. Desenvolva as funções `push()` e `pop()`, criando uma variável de controle manual para rastrear o rastrear o índice exato do topo (`top`).

02 | Fila Circular

Missão: Otimize o uso de memória implementando uma fila sobre um array circular. Desenvolva `enqueue()` e `dequeue()`, controlando independentemente os ponteiros de início (`front`) e fim (`rear`).

“Estruturas de dados não são apenas código; são modelos mentais para organizar a realidade. A escolha entre LIFO e FIFO dita quem espera mais, quem é revertido e quem é atendido primeiro no seu sistema.”

O bom design de software exige escolher a arquitetura correta com propósito deliberado.



Fim da Aula 04

Estrutura de Dados 2

Profª Kadidja Valéria

Bom estudo prático e excelente laboratório!